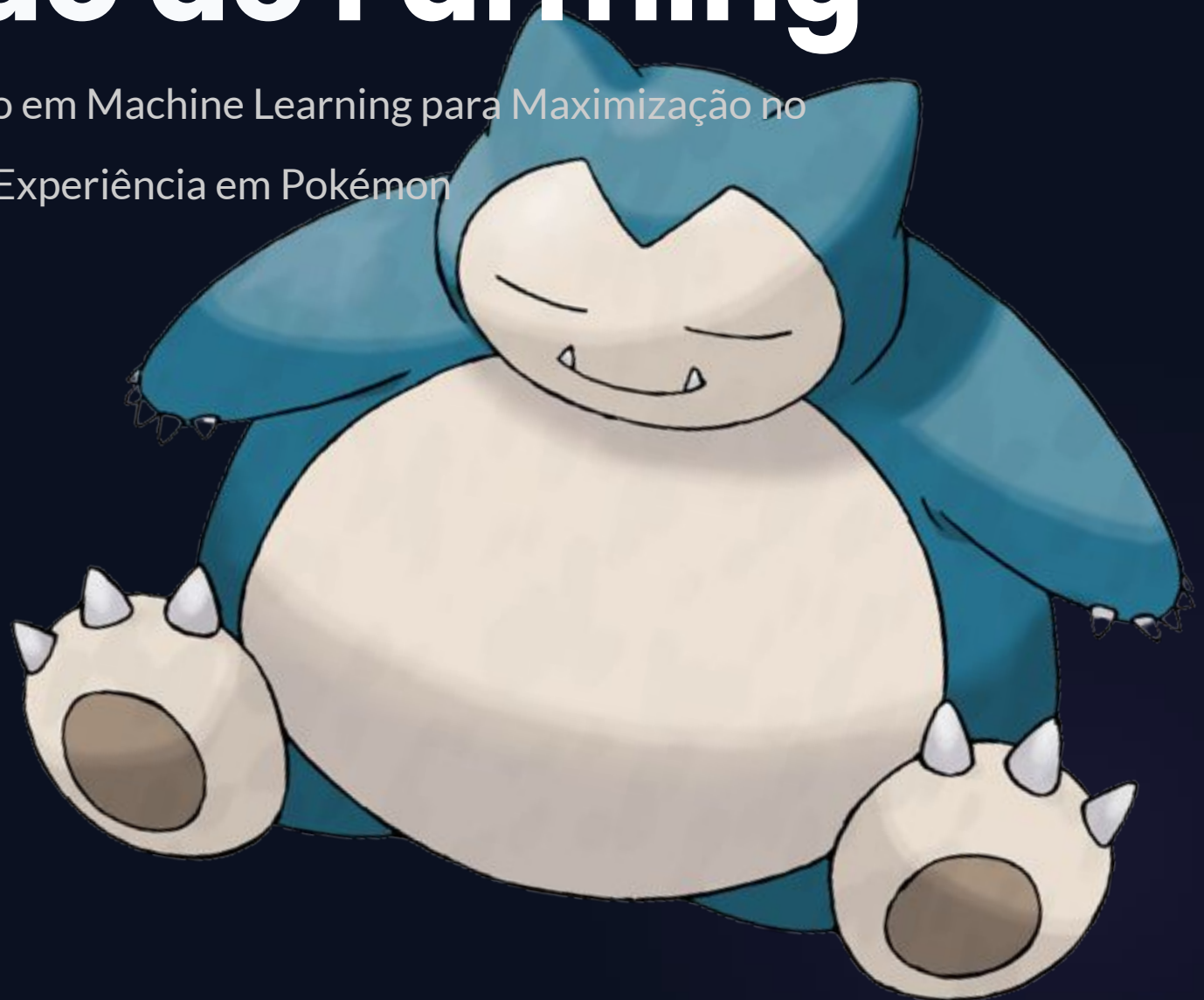


# Otimização de Farming

Um Sistema de Recomendação em Machine Learning para Maximização no  
Ganho de Experiência em Pokémon

Grupo:

Giovanna Oliveira da Silva  
Isabella Vitoria da Silva Raymundo  
Kayo Bacelar de Oliveira  
Leonardo Ambrus de Lima  
Rodrigo da Silva Nogueira





# Recapitulando

A fundação estratégica do sistema de  
recomendação

# Nosso Trabalho

O foco central é **maximizar o ganho de XP** indicando os melhores oponentes de treinamento de forma preditiva.

- **Cenário Competitivo:** Combates 1v1 simulados no Nível 100.
- **Ambiente Determinístico:** Ignora variáveis estocásticas (RNG e críticos).
- **Foco em Atributos:** Baseado inteiramente no choque de atributos brutos e tipagem elementar.





# Nossos Modelos

Por trás do SciKit: Regressão Logística, Random  
Forest e Redes Neurais



# Formalização Espacial do Problema

---



O problema é modelado como uma **Classificação Binária Supervisionada**

$$y \in \{0, 1\} \quad X \in \mathbb{R}^{M \times N}$$

**Espaço amostral:** Matriz X com batalhas  $M=10.000$  simuladas e  $N=14$  dimensões (recursos brutos).

📌 **Target (y):** 1 para vitória do Pokémon aliado (P1); 0 para derrota.

# Regressão Logística: Ordem Preditiva

## ↓ Fluxo de Operações Lineares

**Vetor de Entrada (x):** Instanciação dos status padronizados da batalha.

**Score Linear (z):** Produto escalar dos coeficientes pelo vetor de atributos.

**Sigmóide:** Mapeamento do score contínuo em probabilidade.

**Limiar Lógico:** Comparação binária restrita pelo limiar operacional ( $P \geq 0.85$ ).

O score  $z$  representa a combinação linear projetada sobre o hiperplano de decisão:

$$z = \mathbf{X} \cdot \mathbf{w} + b$$



# Regressão Logística: Função de Ativação



Para achatar o espaço infinito do score  $z$  no intervalo probabilístico estrito de  $[0,1]$ , o scikit-learn aplica a **Função Sigmóide (Curva Logística)**:

$$P(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Se  $z > 0$  : (Vitória)

- Se  $z = 0$  : (Fronteira)

- Se  $z < 0$  : (Derrota)

# Regressão Logística: Log Loss

---

Durante o treino (`.fit()`), os pesos são ajustados minimizando a função de custo de **Entropia Cruzada Binária (Log Loss)** com penalidade  $L_2$  :

$$J(w, b) = -\frac{1}{M} \sum_{i=1}^M \left[ y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] + \frac{1}{2C} \|w\|_2^2$$

• Se  $z > 0$  : (Vitória)

• Se  $z = 0$  : (Fronteira)

• Se  $z < 0$  : (Derrota)





# Random Forest: Amostragem

---

## Bootstrapping

- **Sorteio Aleatório com Reposição:** Cada árvore individual recebe um conjunto próprio de treino de  $M=10.000$  amostras

## Out-of-bag

- Amostras originais que ficam fora do conjunto por conta de duplicatas.
- Funcionam como um conjunto de teste nativo para validar o erro individual da árvore

## Aleatoriedade de atributos

- Cada nó recebe um conjunto aleatório de atributos de tamanho  $\sqrt{N}$



# Random Forest: Impureza de Gini

---

$$\text{Gini ( N\'o )} = 1 - \sum_{i=1}^c (p_i)^2$$

$p_i \rightarrow$  Probabilidade empírica (frequência) de amostras de uma classe pertencentes àquele subconjunto particionado do nó.



## Minimização do Custo do Corte

O Scikit-Learn varre os limiares ( $t$ ) de cada feature e executa o corte binário que resulta no menor custo ponderado dos nós filhos:

$$\text{GiniCorte} = \frac{M_{\text{Esq}}}{M_{\text{Tot}}} \text{Gini ( Esq )} + \frac{M_{\text{Dir}}}{M_{\text{Tot}}} \text{Gini ( Dir )}$$

# Rede Neural: Álgebra Linear do Forward Pass



A arquitetura densa ( $14 \rightarrow 100 \rightarrow 50 \rightarrow 1$ ) processa as predições aplicando encadeamentos sucessivos de transformações lineares filtradas por ativações não lineares:

## Camada Oculta (i)

Multiplicação matricial

$$Z_i = X * W_i + b_i$$

Ativação ReLU

$$A_i = \max(0, Z_i)$$

# Rede Neural: Regra da Cadeia e Backpropagation



$$J(w, b) = -\frac{1}{M} \sum_{i=1}^M \left[ y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

O ajuste adaptativo de pesos ocorre retropropagando o erro da camada final até a camada de entrada. Utiliza-se a **Regra da Cadeia** para derivar a perda em relação a cada matriz de pesos:

$$\frac{\partial J}{\partial W_3} = A_2^T \cdot (\hat{y} - y)$$

## Atualização de Pesos

$$W_{\text{novo}} = W_{\text{antigo}} - \alpha \cdot \text{Gradiente}$$

Esse cálculo iterativo garante que a Rede Neural atue como um **Aproximador Universal de Funções**, moldando-se perfeitamente à não-linearidade da fórmula matemática de dano do simulador.



# Abrindo a “Caixa Preta”

SHAP (SHapley Additive exPlanations)



# Explicabilidade: Abrindo a Caixa Preta com SHAP

---



## Justiça Matemática na Previsão

**SHAP (SHapley Additive exPlanations):** Uma técnica interpretativa poderosa estruturada em fundamentos rígidos da Teoria dos Jogos.

**A Metáfora do Jogo:** Imagine que a previsão algorítmica final é um "jogo de equipes", onde cada atributo (Ataque, Defesa, Velocidade, Tipo) atua como um "jogador" essencial para o resultado.

**A Engenharia:** A fórmula distribui o prêmio (a pontuação do modelo) de forma matematicamente justa, quantificando com precisão quanto cada variável específica empurrou a probabilidade de vitória para cima ou para baixo.

# Valores SHAP: A matemática por trás

## Cálculo do valor SHAP de uma variável

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)]$$

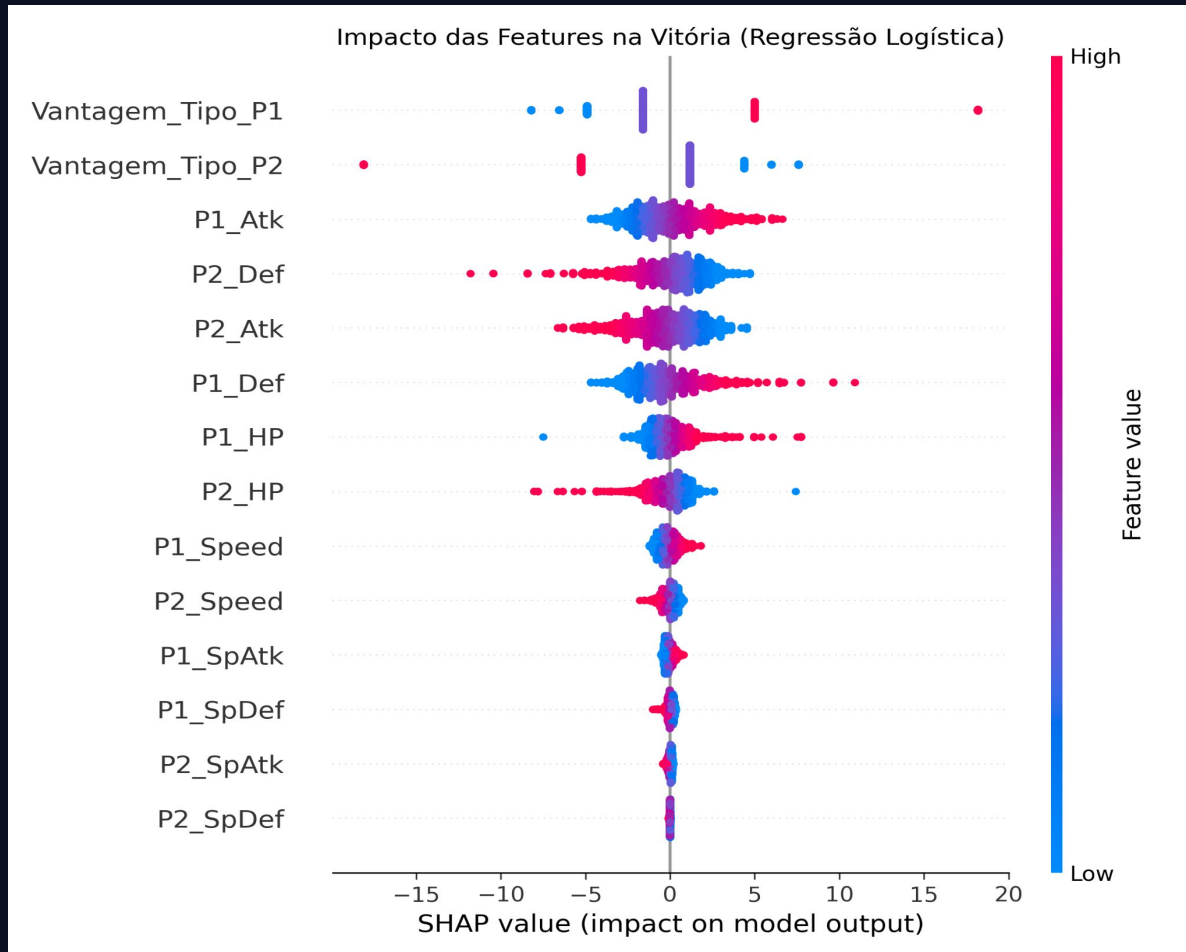
- **N**: conjunto total de features do modelo.
- **S**: uma combinação (subconjunto) de features que não contém a feature i.
- **f(S)**: Previsão do modelo usando apenas as features do subconjunto S. (Como os modelos de ML não aceitam variáveis "faltantes", na prática, os valores das variáveis que não estão em S são substituídos por valores médios de um dataset de referência).
- $\frac{|S|!(|N| - |S| - 1)!}{|N|!}$ : Peso dada a essa combinação específica. Ele garante que a ordem em que as variáveis entram no modelo não distorça o resultado, mantendo o cálculo imparcial.
- **f(S ∪ {i}) - f(S)**: Contribuição marginal. É a diferença na previsão quando adicionamos a feature i ao grupo de variáveis S que já estava presente.

Para encontrar o valor SHAP real de apenas uma variável, a equação exige que seja calculado a contribuição dela em todos os subconjuntos S possíveis, isto é, se um modelo possui N variáveis, existem  $2^N - 1$  combinações.





# Gráficos de Resumo: Regressão Logística



As vantagens de tipo mostram-se as features mais importantes para o modelo, enquanto os atributos especiais e a velocidade são considerados pouco relevantes para o resultado.

Neste modelo, após as vantagens de tipo, o ataque do jogador e a defesa do oponente são considerados ligeiramente mais cruciais no cálculo de vitória do que o poder ofensivo do oponente e a defesa do jogador.

As vantagens de tipo apresentam pontos empilhados verticalmente em determinadas partes do eixo X, pois os valores dessas features não são contínuas, apresentam apenas os seguintes valores: 0, 0.25, 0.5, 1, 2 e 4.



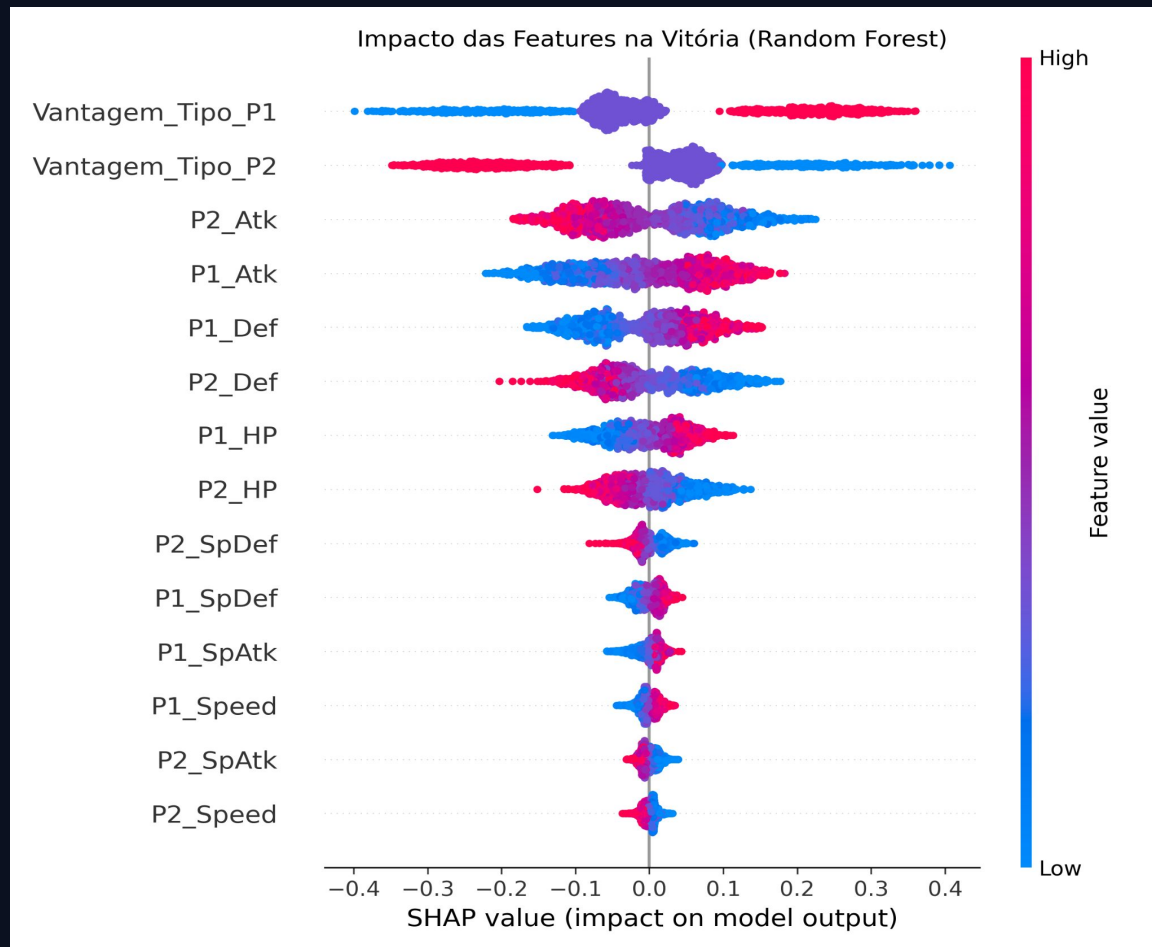
# O Comportamento Rígido (Regressão Logística)



O Eixo X reflete o cálculo bruto (-15 a +20) antes de converter para probabilidade. Nota-se um empilhamento vertical das "Vantagens de Tipo".

Conclusão: O modelo atribui pesos altos e rígidos às vantagens e não cruza informações. Todas as batalhas com a mesma vantagem recebem o exato mesmo valor SHAP, ignorando dinâmicas contextuais de combate.

# Gráficos de Resumo: Random Forest



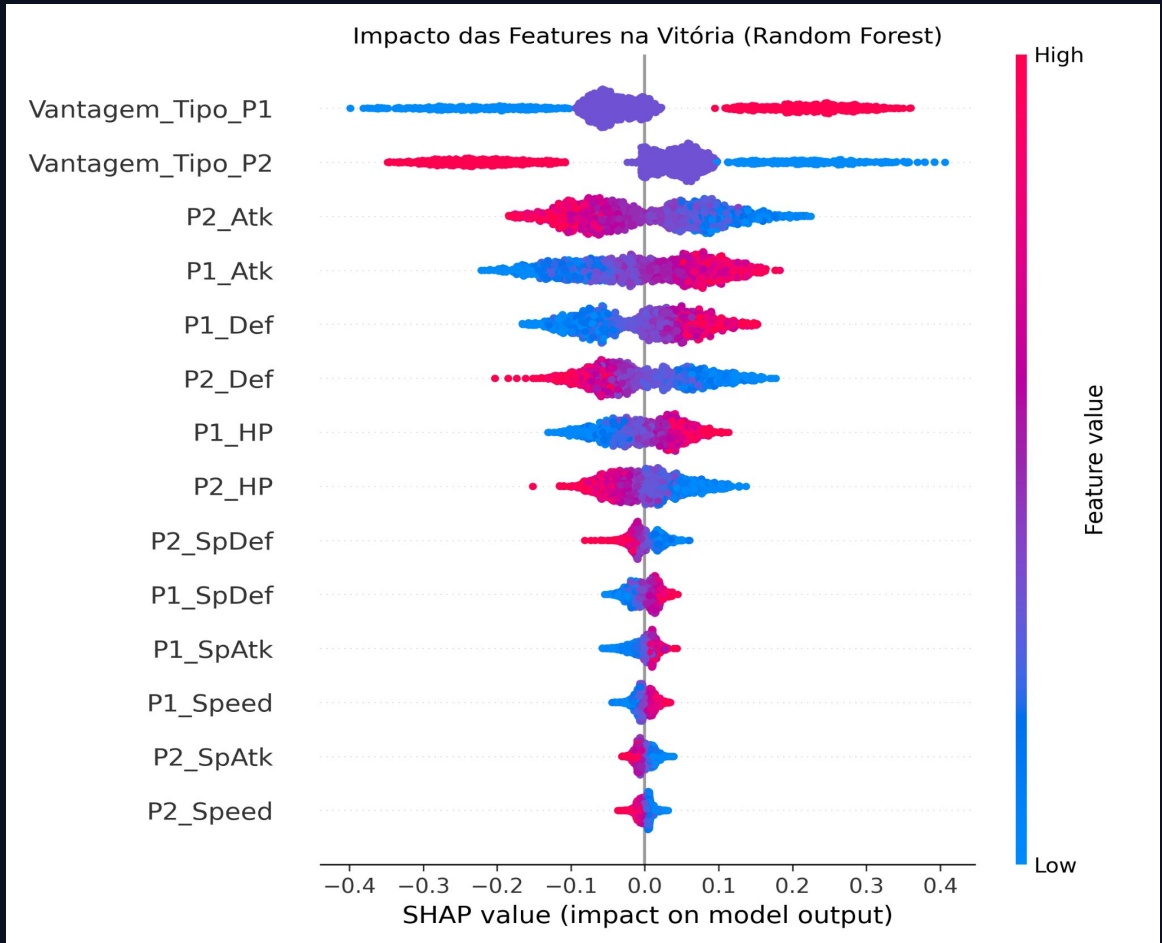
As vantagens de tipo permanecem as features mais importantes para o modelo, em que valores altos para o P1 (pontos vermelhos) puxam a predição fortemente para a direita (vitória do P1), enquanto valores altos para o P2 puxam a predição para a esquerda (vitória do P2).

Em seguida, o modelo valoriza o poder ofensivo mais do que as variáveis defensivas dos Pokémons. Por fim, atributos especiais e a velocidade foram considerados pelo modelo como fatores de menor impacto geral para decidir o vencedor.

# Gráficos de Resumo: Rede Neural (MLP)

Nesse modelo, tivemos que usar uma amostragem reduzida de 50 instâncias para calcular os valores SHAP, pois a Rede Neural exige um alto esforço computacional na extração de explicações matemáticas para suas previsões. Isso explica a menor densidade de pontos nesse gráfico.

Percebe-se que a rede neural manteve a importância dos tipos, apesar de mudar a hierarquia, considerando as vantagens de tipo do oponente mais importante do que as vantagens de tipo do jogador. O padrão de importância dos outros atributos visto nas análises dos modelos anteriores se manteve na Rede Neural.



# A Não-Linearidade Comprovada

---

## #1

VANTAGEM DE TIPO



## #2

ATRIBUTO DE ATAQUE



## A Quebra do Paradigma

**O Julgamento Dinâmico:** No gráfico de Random Forest e Redes Neurais, os pontos de impacto SHAP espalham-se e as cores misturam-se no centro. Isto é a prova matemática de que os algoritmos analisam o "contexto" não-linear de cada combate singular antes de decidirem.

### A Descoberta da Inteligência Artificial:

Existe um forte senso comum de que *"A Velocidade é o atributo mais importante no jogo"*.

Porém, a IA provou que no cenário farm-focado, as **Vantagens de Tipo são absolutas e dominantes**, seguidas de perto pelo Ataque em força bruta. A Velocidade apresentou um impacto real drasticamente menor na equação geral de probabilidade de vitória segura.



# Validação e Performance dos Modelos

Como medimos a qualidade preditiva do  
sistema

# Metodologia de Avaliação — Visão Geral

---

## Como validamos os três Modelos

### Validação Cruzada Estratificada (k=5)

- Dataset dividido em 5 partições (folds) disjuntas
  - Cada fold mantém a mesma proporção de vitórias/derrotas do dataset completo
  - Processo iterativo: 4 folds para treino, 1 fold para teste (rotaciona 5 vezes)
  - Resultado final: média e desvio padrão das 5 execuções
- Nosso dataset tem 10.000 batalhas com distribuição praticamente balanceada. O K-Fold estratificado garante que cada fold mantenha essa mesma proporção.
  - Reprodutibilidade: Fixamos o parâmetro `random_state` para garantir que a divisão dos folds seja sempre idêntica, permitindo reproduzir os mesmos resultados em execuções diferentes.

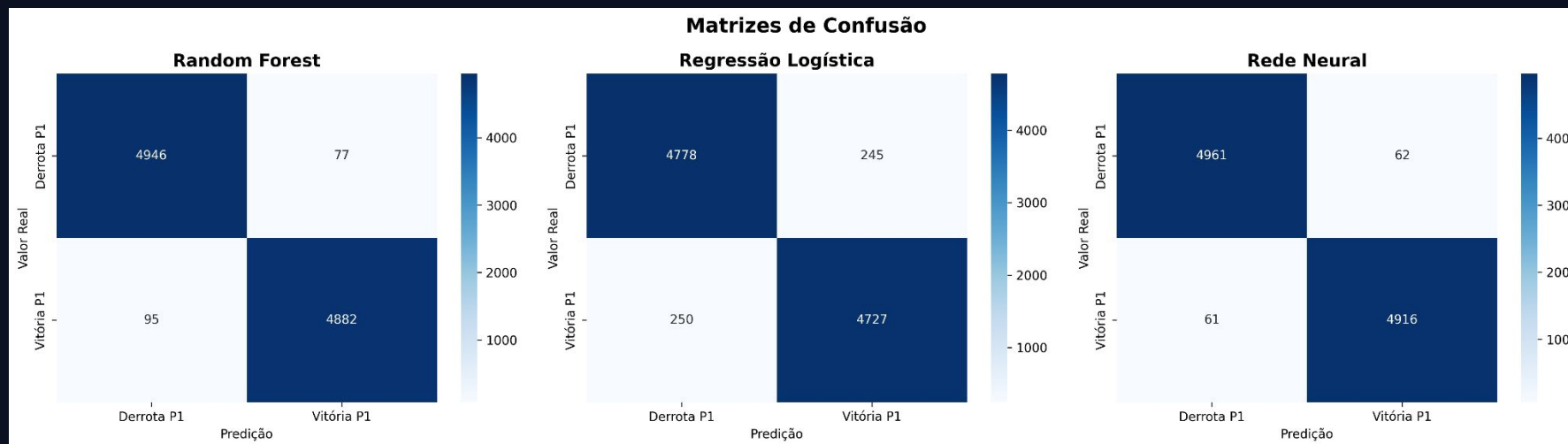
# Escolha das Métricas

## 1. Precision

- $\text{Precision} = \text{VP} / (\text{VP} + \text{FP})$
- "Das batalhas que o modelo previu como vitória, quantas foram vitória de fato?"
- Um falso positivo (FP) = recomendar um oponente que vai te derrotar, jogador perde XP e tempo
- Objetivo: Maximizar para garantir recomendações seguras

## 2. Recall

- $\text{Recall} = \text{VP} / (\text{VP} + \text{FN})$
- "De todas as vitórias reais possíveis, quantas o modelo conseguiu identificar?"
- Um falso negativo (FN) = deixar de recomendar um oponente vencível, oportunidade perdida
- Objetivo: Balancear com Precision para não ser excessivamente conservador



### Vencedor: Rede Neural (MLP)

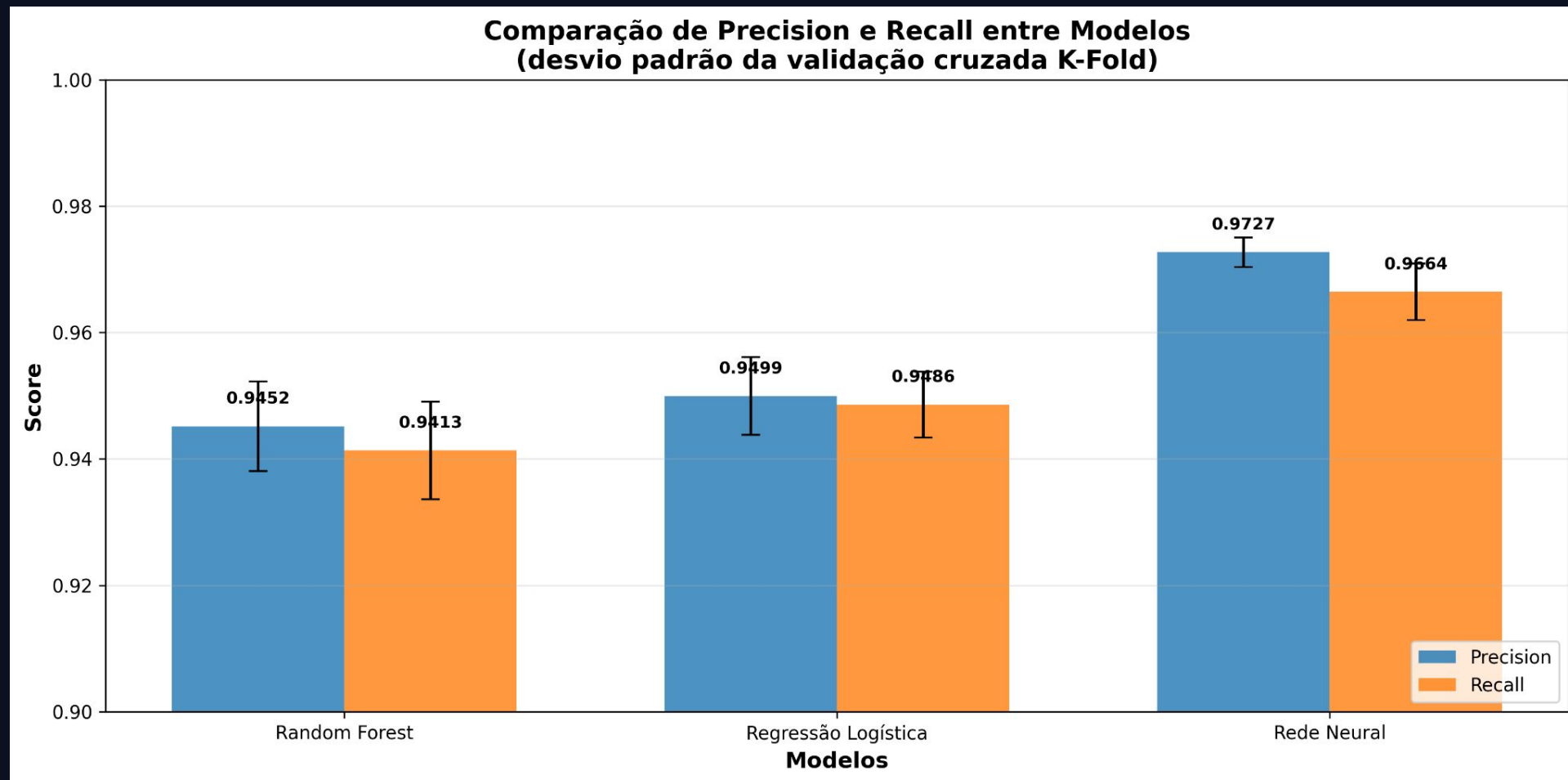
- Melhor Precision (97.27%) e melhor Recall (96.64%)
- Desvio padrão mais baixo dos três, desempenho mais consistente entre os 5 folds

### Regressão Logística: Segundo Lugar

- Resultados muito bons (94.99% e 94.86%)
- Modelo mais simples dos três, mas surpreendentemente eficaz
- Indica que existe uma relação direta entre os atributos e o resultado da batalha

### Random Forest: Terceiro Lugar

- Ligeiramente atrás (94.52% e 94.13%)
- Maior desvio padrão, varia mais entre diferentes divisões dos dados





# Teste de Robustez — Parte 1: Casos Extremos

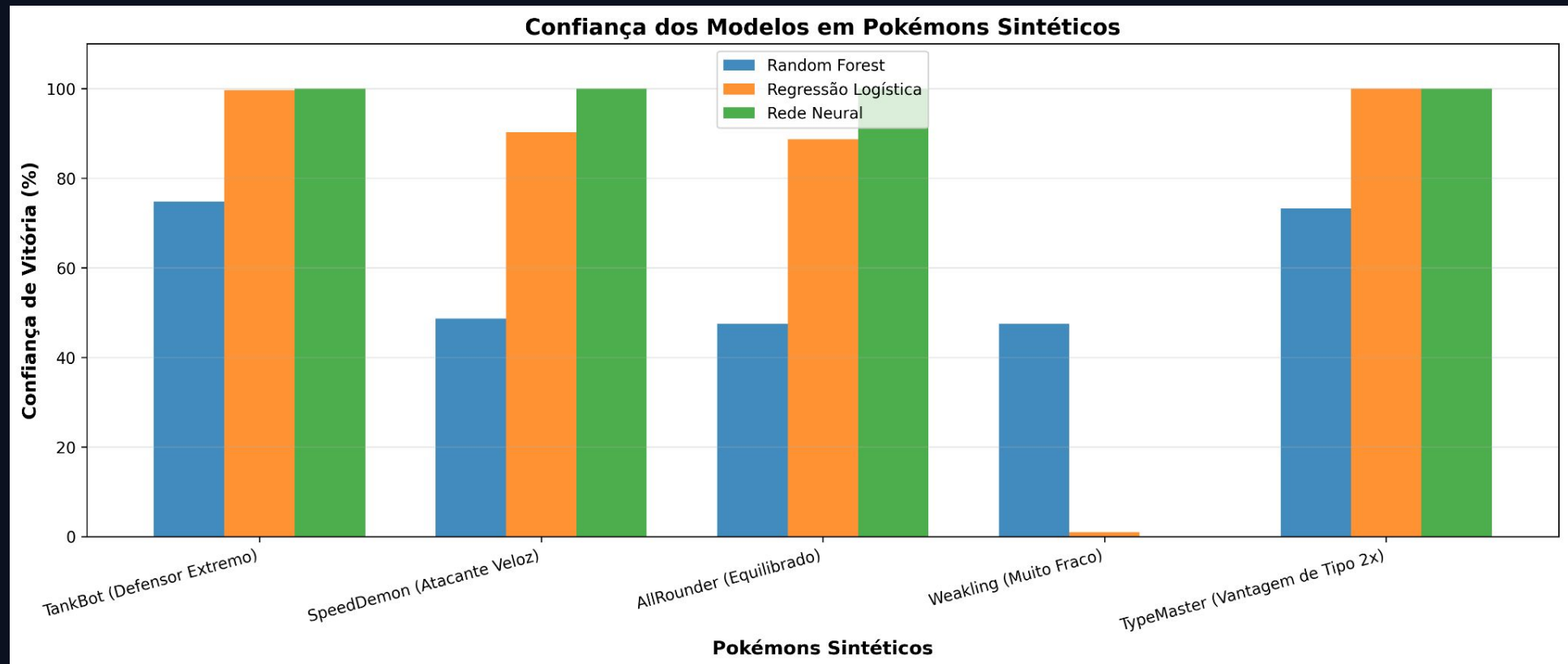
---

## Validação Além do K-Fold: Pokémons Sintéticos Extremos

- Métricas altas ( $>0.94$ ) podem indicar overfitting. Solução: testar com dados fora da distribuição de treino.
- Criamos 5 arquétipos sintéticos extremos com stats que nunca aparecem no dataset real e um oponente padrão com stats fixos
- Como Funciona: Cada Pokémon sintético enfrenta o oponente padrão. Os 3 modelos fazem previsões independentes. Comparamos:
  - Concordância qualitativa: os 3 preveem a mesma classe (vitória/derrota)?
  - Calibração probabilística: qual a confiança de cada modelo?

## O Que Descobrimos nos Casos Extremos

- Concordância de 60%. Os 3 modelos previram a mesma classe em 3 dos 5 casos. MLP operou com saturação, basicamente ou 0% ou 100%. O Random Forest operou de forma mais conservadora. Já a Regressão logística operou entre os outros dois, de forma intermediária, com alta confiança mas sem saturação.
- Limitação Identificada: Stats extremos (20, 200, 250) estão fora da faixa real do dataset (200-400). O Random Forest discordou em 2 dos 5 casos, mas com confiança próxima de 50%, sinal de indecisão causada por valores que ele nunca viu no treino.
- Próximo Passo: Testar com stats dentro da distribuição real, com 30 Pokémons Sintéticos realistas

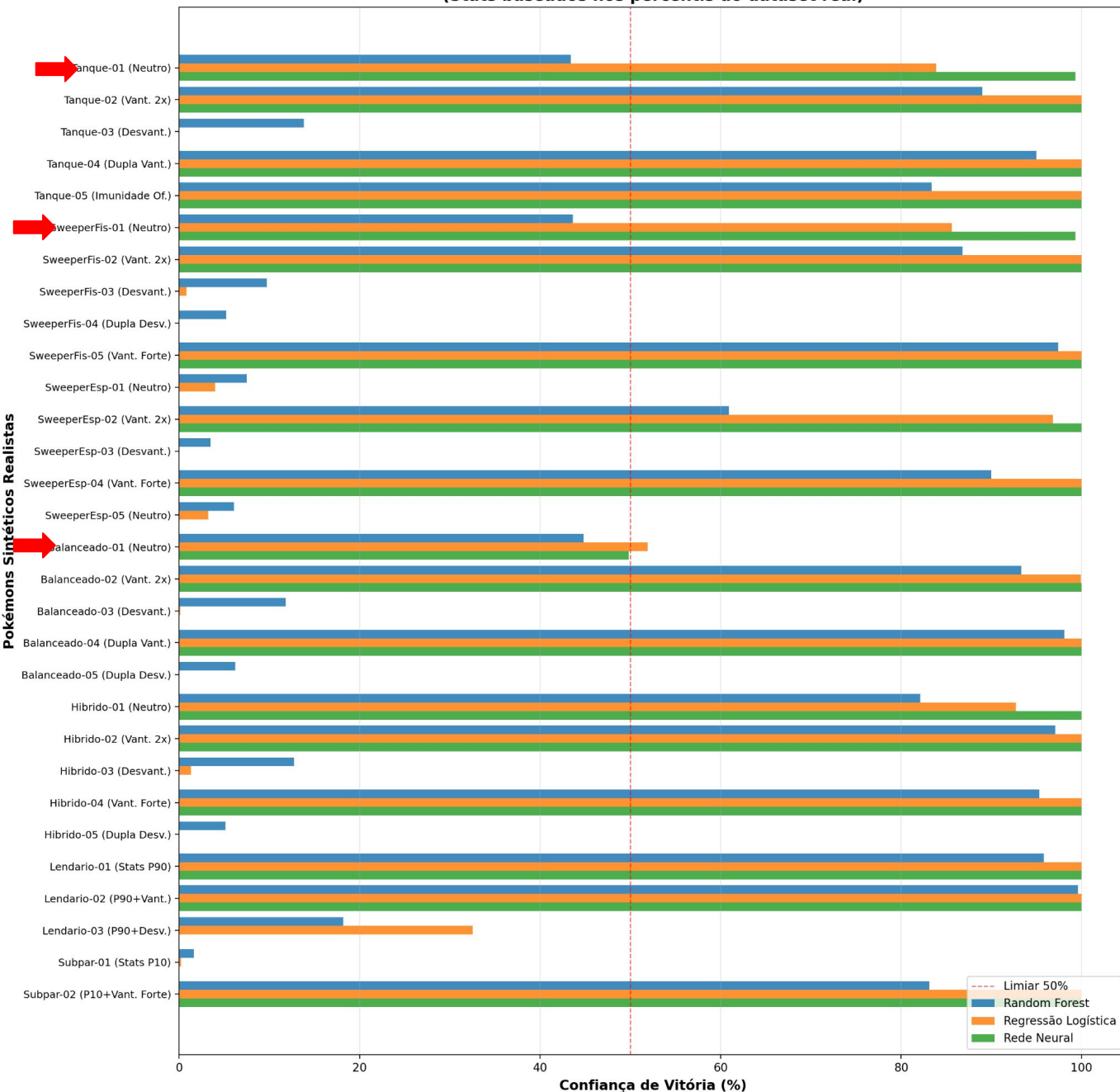


# Teste de Robustez — Parte 2: 30 Pokémons Realistas

---

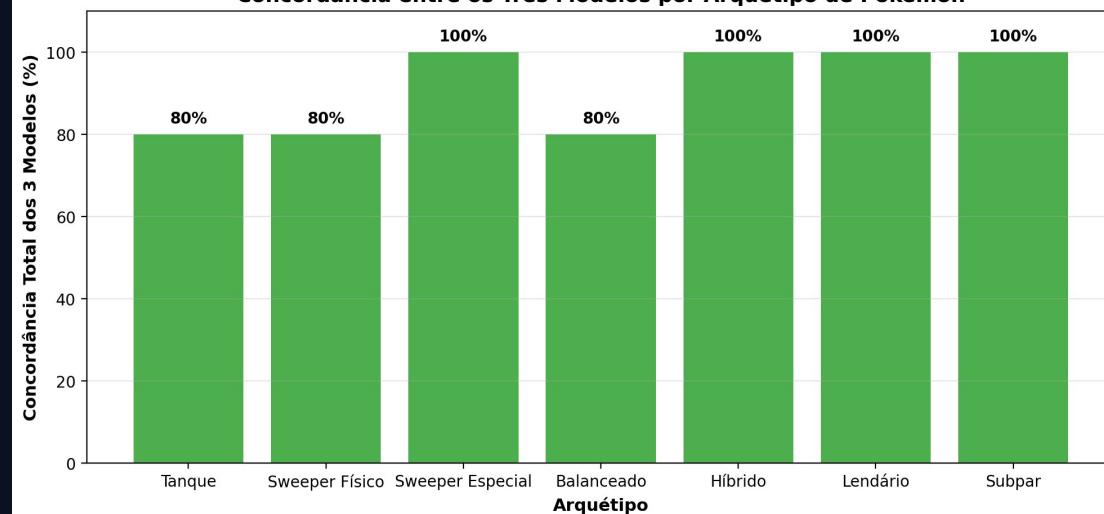
- 30 Perfis Sintéticos Realistas: Criamos Pokémons com stats baseados nos dados reais do dataset
- Concordância saiu de 60% para 90% (27 de 30 casos), com todos os 3 casos de discordância ocorrendo em cenários de tipo neutro (1x/1x).
- Padrão Descoberto: Com vantagem de tipo (2x, 4x), temos a convergência total dos 3 modelos. Quando temos tipos neutros (1x/1x), temos possível divergência, a decisão depende só do equilíbrio de stats.
- Com isso, vemos que a vantagem de tipo é o fator decisivo que une os modelos, quando o tipo é neutro, cada modelo "pensa" diferente.
- 7 Arquétipos:
  - Tanque (HP/Def altos, Atk baixo)
  - Sweeper Físico (Atk/Speed altos)
  - Sweeper Especial (SpAtk/Speed altos)
  - Balanceado (stats medianos)
  - Híbrido (Atk + SpAtk altos)
  - Lendário (stats P90 - muito fortes)
  - Subpar (stats P10 - muito fracos)
- 5 Variações por arquétipo:
  - Neutro (1x/1x)
  - Vantagem leve (2x/1x)
  - Desvantagem (1x/2x)
  - Vantagem forte (4x/0.5x)
  - Combinações extremas
- Total: 30 perfis realistas (maioria com 5 variações, Lendário com 3, Subpar com 2)

**Confiança dos Modelos em 30 Pokémon Sintéticos Realistas**  
(Stats baseados nos percentis do dataset real)



- Os 3 casos de discordância (setas vermelhas) ocorreram exclusivamente em perfis neutros com stats muito próximos do oponente.
- Vantagem de tipo = fator unificador. Quando presente (2x, 4x), concordância total.
- Tipo neutro + stats equilibrados = zona de divergência. Cada modelo "pesa" atributos de forma diferente.
- Isso valida a decisão de expor os 3 modelos na API, eles são genuinamente diferentes, só convergem quando o sinal é óbvio.

**Concordância entre os Três Modelos por Arquétipo de Pokémon**





**Gotta catch em all**

Obrigado pela sua atenção!

